



INDIA.RUNS

Build what next India runs on

Team Name : EkaM

Team Leader Name : Sreenath M Menon

Problem Statement : Recruiters miss the right people because keyword filters can't see who actually fits. Build an AI system that ranks candidates the way a great recruiter would — by *understanding* the role and the person, not matching keywords.

Solution Overview

● What is your proposed solution?

- **TalentSignal** — one JD-agnostic engine that ranks any candidate against any job description **by meaning, not keywords**, rejects fabricated résumés, and explains every decision. Exposed three ways over one engine: a recruiter **Studio UI**, a **REST API**, and an **MCP** server an AI agent can call.
- We built it as a PRODUCTION-GRADE product, not a one-JD hackathon script. The same engine is already LIVE and usable three ways: a recruiter Studio UI, a REST API with interactive Swagger docs, and an MCP server an AI assistant (e.g. Claude Desktop) calls directly — 9 tools + 4 hiring workflows.

● What differentiates your approach from traditional candidate matching systems?

- **Sees beyond keywords.** "Built the recommendation engine serving the homepage" matches "shipped a ranking system" with zero shared keywords — on the real 100K pool, **32% of our top-100 are people a keyword search would have missed entirely.**
- **Trustworthy, not a black box.** Every ranking has a plain-language verdict + the exact evidence; **0 fabricated profiles reached our top-100.**
- **Deterministic & production-real.** No LLM, no GPU, no per-token cost. Ranks 100,000 candidates on a laptop, offline, well within the 5-minute budget. Built like production software: 237 automated tests, a CI quality gate, and byte-identical reproduction.
- **Ready to adopt today** — ingests any format (PDF/DOCX/CSV/JSON/paste); a company can embed it via the REST API; an AI agent can drive it via MCP.

JD Understanding & Candidate Evaluation

● What are the key requirements extracted from the JD?

(Senior AI Engineer)

- **Must-haves:** production ML systems; embeddings / retrieval / ranking / search / vector-hybrid; strong Python; evaluation frameworks (NDCG, MRR, MAP).
- **Disqualifiers:** research-only with no deployment; shallow LLM-demo-only experience; no recent production coding; keyword-stuffing without evidence.

● Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

- **Direct evidence** of production ML/search/ranking work (career text, not just a skills list) is dominant. Then seniority-band fit, then behavioral signals (activity, responsiveness, availability) as tie-breakers.
- Every JD is parsed into a **weighted requirement model** (must-have / nice-to-have / disqualifier). Each requirement is matched to the candidate's evidence by **semantic similarity (sentence embeddings) + lexical overlap** — so a strong fit described in different words is still surfaced.

Ranking Methodology

● How does your system retrieve, score, and rank candidates?

1. **Ingest** any JD + any résumés (PDF/DOCX/TXT/CSV/JSON/paste) into a canonical form.
2. **Parse** the JD into weighted requirements.
3. **Match** each requirement to candidate evidence — hybrid **semantic + lexical**.
4. **Audit** each candidate for internal contradictions (honeypot defense).
5. **Score & rank** with one law; optional cross-encoder rerank on the shortlist.

● What models, algorithms, or heuristics are used?

- Sentence-embeddings (all-MiniLM-L6-v2, run offline/CPU) for meaning; lexical token overlap for exactness; a role-independent consistency auditor; a schema-driven behavioral model.
- **No LLM in the ranking path** — deterministic, reproducible, in-budget.

● How are multiple candidate signals combined into a final ranking?

$$\text{final} = R^{1.1} \times (0.45 + 0.55 \cdot Q) \times \text{soft_penalty} \times \text{hard_veto}$$

- **R** = relevance to this JD's own requirements (dominant; the 1.1 exponent punishes weak relevance harder than it rewards).
- **Q** = quality blend {seniority, logistics, behavioral, trust}, floored at 0.45 so quality tunes but never overrides relevance.
- **soft_penalty** = graded risk/availability penalties; **hard_veto** = 0 only for an internally-impossible profile or a true disqualifier with no counter-evidence.
- One law, parameterized entirely by the parsed JD → the same code ranks an AI JD, a sales JD, or a designer JD with no per-role branches.

Explainability & Data Validation

● How are ranking decisions explained?

- Every candidate gets a **plain-language verdict** (Strong / Worth-a-look / Weak), a **Matched ✓ / Missing ✗ skills panel** against the JD's must-haves, and a 1-2 sentence reasoning drawn from the candidate's own profile.

● How do you prevent hallucinations or unsupported justifications?

- An **explanation audit** fails the build if any reasoning claim isn't grounded in the candidate's actual profile. Reasoning uses the candidate's own evidence terms — it never invents a skill or employer. (Verified: **0 hallucination warnings.**)

● How does your solution handle inconsistent, low-quality, or suspicious profiles?

- A **role-independent consistency auditor** flags impossible profiles by contradiction, not keywords (e.g. expert skill with 0 months, tenure longer than the whole career).
- **0 fabricated profiles reached our top-100** on the real 100K pool.
- Measured **résumé-attack resistance: 100%** on our semantic engine (keyword stuffing, prompt injection, fabricated experience, impossible tenure) vs 78% on a keyword baseline.
- **Tested against a purpose-built adversarial suite.** We built a test harness generating labeled candidates in 7 archetypes — including honeypots (impossible / keyword-stuffed), behavioral-twins (great on paper, dead engagement), and paraphrase-ideal fits (right person, ZERO shared keywords). Iterating against these suites is precisely how we drove the honeypot rate in the top-10 to 0.000 and taught the engine to catch fakes by contradiction, not keywords.

End-to-End Workflow

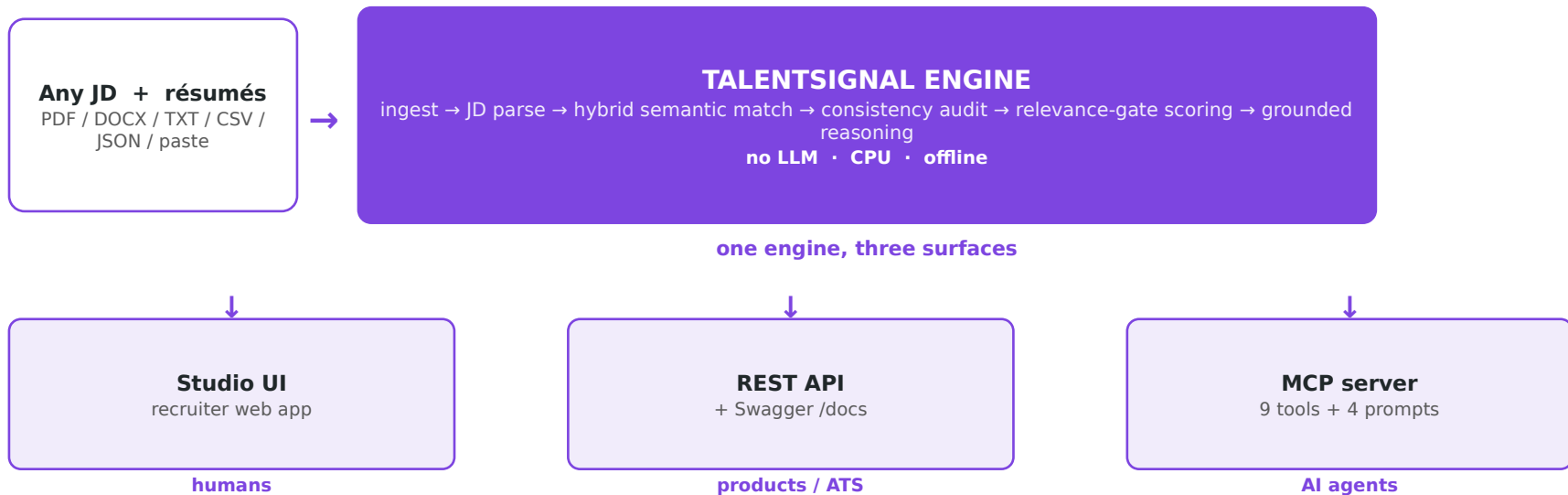
- What is the complete workflow from JD input to ranked candidate output?



Ranked, explained output — delivered through any of the three live surfaces: the Studio UI (a recruiter clicks), the REST API (a product embeds), or MCP tools (an AI assistant calls). Same engine, same result, everywhere.

The exact same pipeline runs for the hackathon's 100,000 candidates and for a recruiter pasting 5 résumés — that's what makes it a product, not a one-off script.

System Architecture



A precomputed embedding index is built offline (once) so the ranking step loads only numpy arrays — offline, CPU-only, in budget.

Results & Performance

- What results or insights demonstrate ranking quality?
- How does your solution meet the challenge's runtime and compute constraints?

Metric	Result	What it proves
Rescued by meaning	32 / 100 top candidates a keyword search misses	Finds people keyword filters can't see
Honeypots in top-100	0	Rejects fabricated / impossible profiles
Résumé-attack resistance	100% (semantic) vs 78% (keyword)	Hard to game
Ranking-quality composite (our labeled eval)	0.95 spine / 0.97 hybrid	Strong NDCG/MAP/P@10 across roles
Official validator	PASS	Meets the exact submission format

- **Validated with a real evaluation framework** (not eyeballing): a test harness across 6 roles (AI/Search, Backend, Data, Design, Product, Sales) with labeled candidates (graded 0-5) in 7 archetypes — proving the engine is JD-agnostic, not tuned to one JD. Measured with NDCG@10/@50, MAP, P@10, honeypot-rate and a zero-keyword paraphrase suite: composite 0.95 / 0.97, honeypot@top-10 0.000, paraphrase fits recovered 10 / 10, cross-JD top-10 overlap ~0. 237 automated tests + a CI quality gate on every commit.
- Ranks the full **100,000 candidates in ~30s (spine) / ~67s (hybrid)** — well under the 5-minute budget.
- **CPU-only, network OFF, no hosted LLM, ~450 MB** — fits ≤16 GB RAM / ≤5 GB disk.
- **Deterministic**: the submission reproduces **byte-identical** offline.

Technologies Used

Technology	Why chosen
Python 3.11	Portable, standard, easy to reproduce.
sentence-transformers (all-MiniLM-L6-v2)	Meaning-based matching; small, runs offline on CPU — no GPU, no API.
NumPy	The only dependency at rank time — loads the precomputed index fast, deterministically.
Stdlib http.server	Studio UI, REST API, and MCP server need no web framework — light, offline, easy to host.
OpenAPI + Swagger UI	Interactive, self-documenting REST API for integrators.
MCP (Model Context Protocol)	Lets an AI agent call the engine — 9 tools + 4 hiring workflow prompts.
Git LFS	Ships the precomputed embedding index for exact reproduction.
pytest + CI	237 automated tests + a quality gate on every commit — built like production software, not a one-off script.

Key choice: no LLM / no GPU in the ranking path — because a system that calls an LLM per candidate can't rank 100K in a 5-minute CPU budget in production. Meaning comes from precomputed embeddings; the ranking is fast, cheap, and reproducible.

Engineered as a product, not a submission: one clean engine facade behind three live surfaces (UI · REST · MCP), universal ingest, an evaluation framework across 6 roles, and byte-identical reproducibility — the things a company needs to actually adopt it, not just a script that produces one CSV.

Submission Assets

- **GitHub repository:** <https://github.com/sreenathmmenon/talentsignal-ai> (public)
- **Ranked output (top-100):** [outputs/final_submission.xlsx](#) (and [.csv](#)) — official validator passes.
- **Demo video:** <https://youtu.be/pcIS4QzkD-o>
- **Live demo (Studio UI):** <https://talentsignal-production.up.railway.app>
- **Live REST API + Swagger docs:** <https://talentsignal-api-production.up.railway.app/docs>
- **Reproduce in one command:** `make reproduce` (byte-identical, offline, CPU, ~70s).
- **Tested:** 237 automated tests; CI quality gate on every commit.
- **Also usable as MCP tools for AI assistants (Claude Desktop)** — 9 tools + 4 workflows.



INDIA.RUNS

Build what next India runs on

THANK YOU